Praktikum JavaScript & TypeScript

Soal Modul 2 - Perulangan dan Fungsi

1. Tugas

Kamu membantu tim IT sebuah bank digital dan sebuah rumah sakit. Data mutasi rekening sudah tersedia di bagian `PEMANGGILAN` pada starter. Angka positif berarti uang masuk, angka negatif berarti uang keluar.

> **⚠ Perhatian**
>
> 1. Salin `Soal/starter.js` ke folder `Jawaban/`, misalnya `Jawaban/soal-js2.js`.
> 2. Lengkapi fungsi bertanda `TODO`. Jangan mengubah bagian `PEMANGGILAN`.
> 3. Jalankan `node Jawaban/soal-js2.js`. Output harus sama persis dengan Contoh Output.
> 4. Soal bertanda **(jelaskan)** dijawab sebagai komentar di file jawaban, di bawah baris `// JAWABAN ...`.
> 5. Method array selain `push` dan `length` (misalnya `map`, `reduce`, `forEach`) belum boleh dipakai. Semua perulangan ditulis manual.

1.1. Beginner (0.3)

1. **Saldo akhir.** Lengkapi `saldoAkhir(saldoAwal, mutasi)` memakai `for` klasik. Fungsi mengembalikan saldo setelah semua mutasi diterapkan.
2. **Ringkasan mutasi.** Lengkapi `ringkasMutasi(mutasi)` memakai `for...of`. Fungsi mengembalikan string dengan format persis `Masuk: X, Keluar: Y, Transaksi: Z`. `Keluar` ditulis sebagai angka positif.
3. **Kode temanmu.** Fungsi `hitungPPN` di starter selalu mengembalikan `undefined`.
   a. **(jelaskan)** Mengapa?
   b. Perbaiki dengan **dua** cara berbeda. Tulis cara yang tidak dipakai di komentar, dan pakai salah satunya di kode.

1.2. Intermediate (0.3)

1. **Target tabungan.** Lengkapi `bulanMencapaiTarget(saldoAwal, setoranBulanan, bungaPersenTahunan, target)`. Setiap bulan, setoran ditambahkan dulu, lalu bunga bulan itu dihitung dari saldo baru: `bunga = Math.floor(saldo * bungaPersenTahunan / 100 / 12)`. Fungsi mengembalikan jumlah bulan sampai saldo **minimal** sama dengan target.
   - Jika saldo awal sudah mencapai target, kembalikan `0`.
   - Jika target tidak akan pernah tercapai, kembalikan `-1`. **(jelaskan)** Kondisi apa yang membuat target tidak pernah tercapai, dan apa yang terjadi pada program jika kondisi ini tidak dicek?
2. **Kode temanmu.** Fungsi `totalTagihan` menjumlahkan biaya perawatan pasien, tetapi hasilnya `NaN`.
   a. **(jelaskan)** Ada **dua** kesalahan di perulangannya. Sebutkan keduanya dan jelaskan mengapa hasilnya `NaN`, bukan error.
   b. Tulis ulang fungsi memakai `for...of`. **(jelaskan)** Mengapa versi `for...of` tidak bisa terkena kesalahan yang sama?
3. **Antrean poli.** Lengkapi `posisiPertama(antrean, kodePoli)`. Fungsi mengembalikan posisi pertama (mulai dari 1) pasien dengan kode poli tersebut, atau `-1` jika tidak ada. Perulangan harus berhenti begitu data ditemukan. **(jelaskan)** Mengapa `for...of` kurang cocok untuk soal ini dibandingkan `for` klasik?

1.3. Advanced (0.4)

1. **Jadwal cicilan pinjaman.**

a. Lengkapi `jadwalCicilan(pokok, bungaPersenPerBulan, cicilanPerBulan)`. Fungsi mengembalikan **array of string**, satu string per bulan, dengan format `Bulan N: bunga B, bayar P, sisa S`. Aturan setiap bulan:
   a. `bunga = Math.floor(sisa * bungaPersenPerBulan / 100)`, lalu bunga ditambahkan ke sisa.
   b. Pembayaran sebesar `cicilanPerBulan`. Jika sisa lebih kecil dari cicilan, bayar sebesar sisa saja.
   c. Ulangi sampai sisa menjadi 0.

   Jika cicilan bulan pertama tidak lebih besar dari bunga bulan pertama, pinjaman tidak akan pernah lunas. Kembalikan `["TIDAK AKAN LUNAS"]`.

b. **(jelaskan)** Mengapa fungsi ini mengembalikan array string, bukan langsung `console.log` di dalam fungsi? Sebutkan minimal dua keuntungannya.

2. **Biaya admin penarikan.**
   a. Lengkapi `biayaBertingkat(nominal)`: penarikan sampai Rp 100.000 gratis, sampai Rp 1.000.000 biayanya Rp 2.500, di atas itu Rp 5.000.
   b. Lengkapi `totalBiayaAdmin(mutasi, aturanBiaya)`. Fungsi menjumlahkan biaya admin untuk setiap mutasi **keluar** saja. Parameter `aturanBiaya` adalah fungsi yang menerima nominal penarikan (positif) dan mengembalikan biayanya. Tiga aturan (`biayaFlat`, `biayaPersen`, `biayaBertingkat`) dipakai bergantian di bagian `PEMANGGILAN`.
   c. **(jelaskan)** Temanmu mengusulkan versi lain: `totalBiayaAdmin(mutasi, jenis)` dengan `jenis` berupa string `"flat"`, `"persen"`, atau `"bertingkat"`, lalu `if` di dalam fungsi. Bank lalu menambah aturan baru “gratis untuk nasabah prioritas”. Bandingkan berapa banyak kode yang harus diubah pada versimu dan versi temanmu.

2. Contoh Output

Contoh Output

```
== Beginner 1 ==
1305000
0
== Beginner 2 ==
Masuk: 1575000, Keluar: 520000, Transaksi: 6
Masuk: 0, Keluar: 0, Transaksi: 0
== Beginner 3 ==
11000
2750
== Intermediate 1 ==
6
4
0
-1
== Intermediate 2 ==
525000
0
== Intermediate 3 ==
2
3
-1
```

Contoh Output

```
== Advanced 1 ==
Bulan 1: bunga 20000, bayar 300000, sisa 720000
Bulan 2: bunga 14400, bayar 300000, sisa 434400
Bulan 3: bunga 8688, bayar 300000, sisa 143088
Bulan 4: bunga 2861, bayar 145949, sisa 0
TIDAK AKAN LUNAS
== Advanced 2 ==
7500
2600
5000
0
```

3. Pengumpulan

⚠ Perhatian

- Kode soal: `Jawaban/soal-js2.js`. Jawaban **(jelaskan)** ditulis sebagai komentar di bawah baris `// JAWABAN ...` yang sesuai.
- Coba Sendiri dari HO dikerjakan di fungsi `cobaSendiri()` pada file yang sama. Jalankan dengan `node Jawaban/soal-js2.js coba`.
- Jalankan `npm run selesai -- --durasi "..." --macet "..."`, lalu minta asisten memeriksa.